# PIZZA STORE

## Full-Stack Web Application

Built with the MERN Stack

| | |
|---|---|
| **Project Type** | Capstone Project Report |
| **Submitted By** | R Bhaskar Naik |
| **Guided By** | Abhishek Sharma |
| **Date** | March 2026 |

# Table of Contents

Contents…….. Page No

1. Problem Statement ..... 3
2. Goals and Objectives ..... 4
3. Abstract ..... 5
4. Introduction ..... 5
5. Objectives ..... 5
6. Technology Stack ..... 5
7. System Architecture ..... 7
8. Database Design — ER Diagram……..8
8.1. Data Flow Diagrams……9 & 10
9. Features Implemented ..... 11
10. Client-Side Validation ..... 12
11. Routing ..... 12
12. Frontend–Backend Integration……13
13. API Endpoints Reference ..... 13
14. Testing Strategy ..... 15
15. Application Workflow ..... 15
16. Application Screenshots ..... 16
17. Conclusion ..... 21

## 1. Problem Statement ..... 3

Pizza Store (Pizzeria) is a full-stack web application where customers can order pizzas of their choice online. The application serves two types of users: Admin and Customer.

**Admin Requirements:**

- Admin should be able to login and logout of the application.
- Admin should be able to perform CRUD operations on menu items.
- Admin can accept or reject orders from users.
- Admin should be able to send messages or pop-up notifications to users on order status updates.
- Admin should be able to generate the bill for a particular user.
- Admin should be able to view the monthly revenue of the shop.

**Customer Requirements:**

- Customer should be able to register, login, and logout of the application.
- Customer should be able to browse menu categories including pizza, sides, beverages, combos, new launches, and bestsellers.
- Customer should be able to select multiple items from the same or different categories.
- Customer should be able to place or cancel orders.
- Once an order is placed, the customer should receive a message or pop-up notification about the order status.
- Customer should be able to view the bill amount along with payment options.
- Customer should be able to see the available mode of delivery.

## 2. Goals and Objectives ..... 4

The primary goal of this project is to design and develop a fully functional, scalable, and user-friendly online pizza ordering platform using the MERN stack. The specific objectives of this project are:

1. **Design and Implement a Full-Stack Application:** Build a complete pizza ordering system with separate customer and admin interfaces.
2. **Enable Menu Browsing:** Allow customers to view menu items, categories, prices, and images with search and filter functionality.
3. **Implement Secure Authentication:** Provide JWT-based login and registration for all users to ensure data security and session management.
4. **Manage Cart and Orders:** Enable cart management and order placement with real-time status notifications via Socket.IO.
5. **Build an Admin Management Dashboard:** Create an admin panel to manage products, categories, users, orders, and view monthly revenue.
6. **Develop a Responsive UI:** Build a responsive interface using React and Tailwind CSS that works across all device sizes.
7. **Integrate Payment Gateway:** Support Cash, Card, and UPI payment options using Razorpay.
8. **Ensure Application Quality:** Perform unit and end-to-end testing to validate the correctness and reliability of both frontend and backend components.

## 3. Abstract ..... 5

The **Pizza Store** system is a full-stack web application developed to simplify the online pizza ordering process. The platform enables customers to explore available pizza items, create an account, log in securely, add products to their shopping cart, and place orders conveniently. An administrative dashboard is included to manage menu categories, products, users, and customer orders efficiently.

The user interface is built using **React** with **Redux** for state management and **Tailwind CSS** for styling. Server-side operations use **Node.js + Express.js**. Data is stored in **MongoDB**. Secure login uses **JWT authentication**. Real-time alerts use **Socket.IO** and payments via **Razorpay**.

## 4. Introduction ..... 5

In recent years, online food ordering platforms have become increasingly popular. The **Pizza Store** web application provides an efficient system for ordering pizzas online. Customers can browse menu items, register, log in, and place orders through a simple interface. The application uses React for the frontend, Node.js and Express.js for the backend, and MongoDB for data storage. The main objective is to demonstrate a modern, scalable, interactive, and user-friendly full-stack application.

## 5. Objectives ..... 5

| | |
|---|---|
| **Design & Implement** | Build a complete full-stack pizza ordering app with customer and admin interfaces. |
| **Menu Browsing** | Allow customers to view menu items, categories, prices, and images. |
| **Secure Authentication** | Provide JWT-based login and registration for all users. |
| **Cart & Orders** | Enable cart management and order placement with real-time status updates. |
| **Admin Management** | Create admin dashboard to manage products, categories, users, and orders. |
| **Responsive UI** | Build a responsive interface using React and Tailwind CSS. |
| **Real-Time Notifications** | Integrate Socket.IO for instant admin notifications on new orders. |
| **Payment Integration** | Support Cash, Card, and UPI payments via Razorpay. |

## 6. Technology Stack ..... 5

| Layer | Technology | Purpose |
|---|---|---|
| Frontend | React 18 | UI component framework with fast builds |
| State Management | Redux Toolkit | Centralized application state management |
| Styling | Tailwind CSS | Utility-first responsive styling |

| | | |
|---|---|---|
| Backend | Node.js + Express.js | RESTful API server and business logic |
| Real-Time | Socket.IO | Live order notifications and alerts |
| Database | MongoDB + Mongoose | NoSQL storage and object modelling |
| Authentication | JSON Web Token (JWT) | Stateless secure user authentication |
| Payment | Razorpay | UPI, card, and wallet payment gateway |
| Security | Helmet + express-rate-limit | HTTP headers and rate limiting |
| Email | Nodemailer | Password reset notification emails |
| Testing | Mocha + Chai + Supertest | Backend unit and integration testing |

## 7. System Architecture ..... 7

The Pizza Store platform follows a **three-layer (MVC) architecture**, ensuring clear separation of concerns.

### Presentation Layer (Frontend)

Built with React 18 + Vite, Redux Toolkit for global state, and Tailwind CSS. Users interact with Home, Login/Register, Menu, Cart, My Orders, and Admin Panel pages. React Router v6 handles client-side navigation.

### Application Layer (Backend)

Node.js + Express.js handles all REST API requests. Controllers manage auth, menu, cart, orders, payments, and messages. Middleware provides JWT auth, role-based access, input validation, rate limiting, and error handling.

### Data Layer (Database)

MongoDB stores all data via Mongoose ODM. Collections: Users, Categories, MenuItems, Cart, Orders, Addresses, Payments, Coupons, and Messages. REST APIs handle frontend-backend communication; Socket.IO handles real-time events.

## 8. Database Design — ER Diagram ..... 7

The application uses MongoDB with Mongoose ODM. Key relationships:

- CATEGORIES → MENU_ITEMS: One-to-many (via categoryId FK)
- USERS → ADDRESSES: One-to-many (via userId FK)
- USERS → CART: One-to-one (via userId FK)
- USERS → ORDERS: One-to-many (via userId FK)
- ORDERS → PAYMENTS: One-to-one (via orderId FK)
- ORDERS → MESSAGES: One-to-many (admin notification messages)

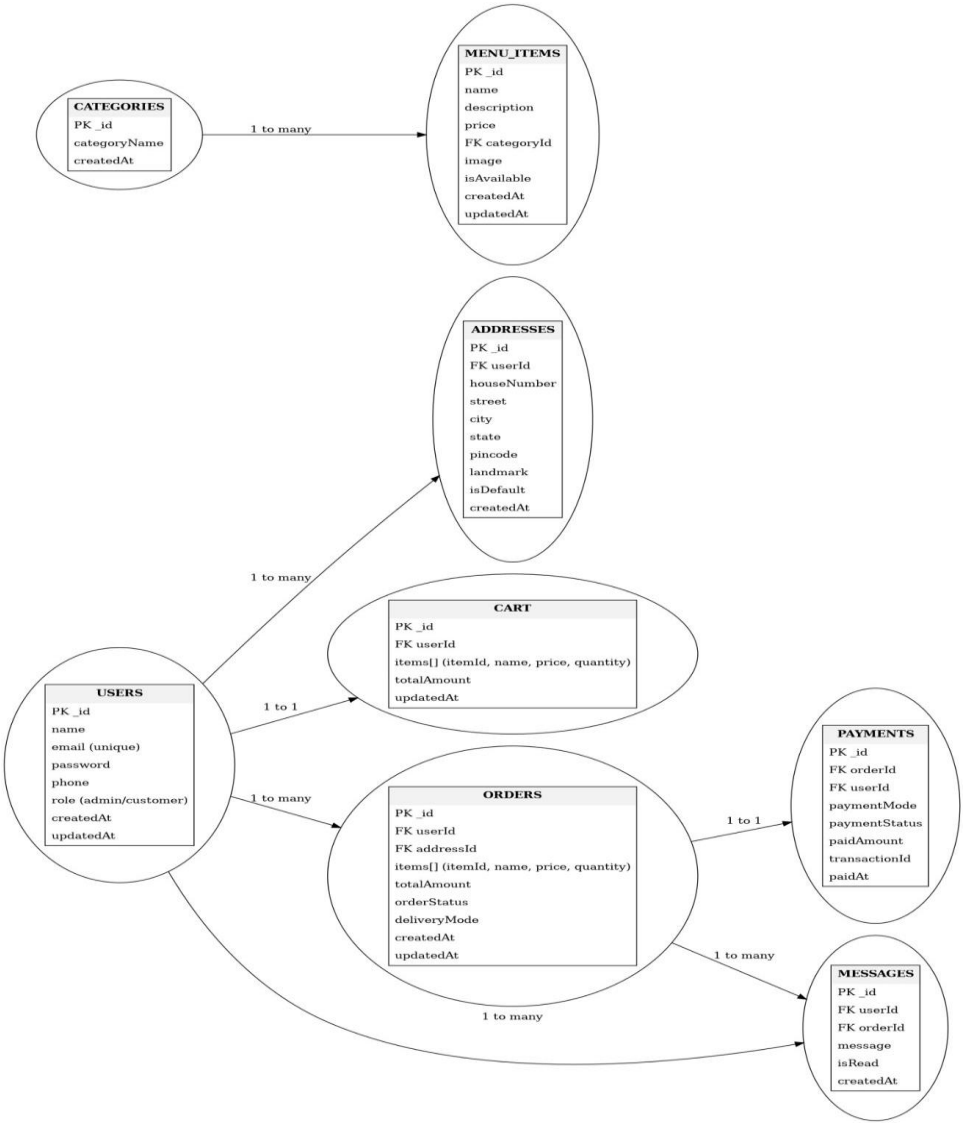


*Figure 1: Entity-Relationship Diagram of Pizza Store Database*

## 6.1 Data Flow Diagram — Level 0 (Context Diagram)

The Level 0 DFD (Context Diagram) shows the Pizza Store system as a single process and identifies all external entities that interact with it, along with the high-level data flows between them.

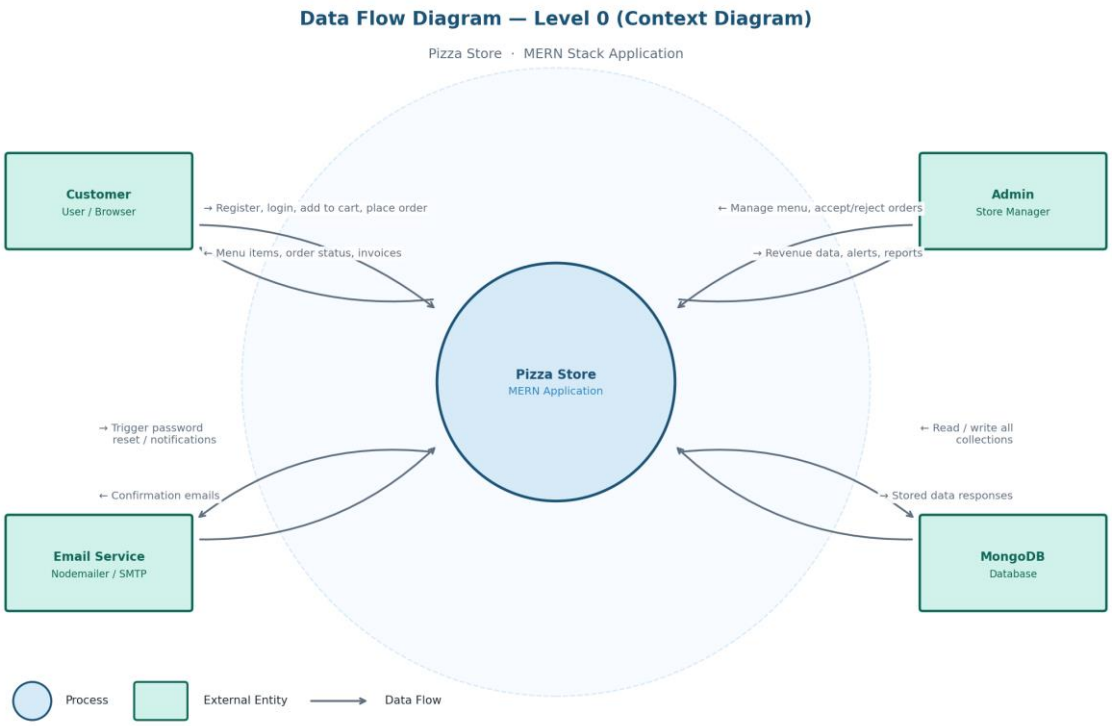


*Figure 2: DFD Level 0 — Context Diagram (Pizza Store)*

## 6.2 Data Flow Diagram — Level 1 (Internal Processes)

The Level 1 DFD expands the Pizza Store system into its six internal processes: Authentication (P1), Browse & Cart (P2), Order Processing (P3), Address Management (P4), Admin Management (P5), and Notifications (P6). It also shows all six data stores and the data flows between processes, stores, and external entities.

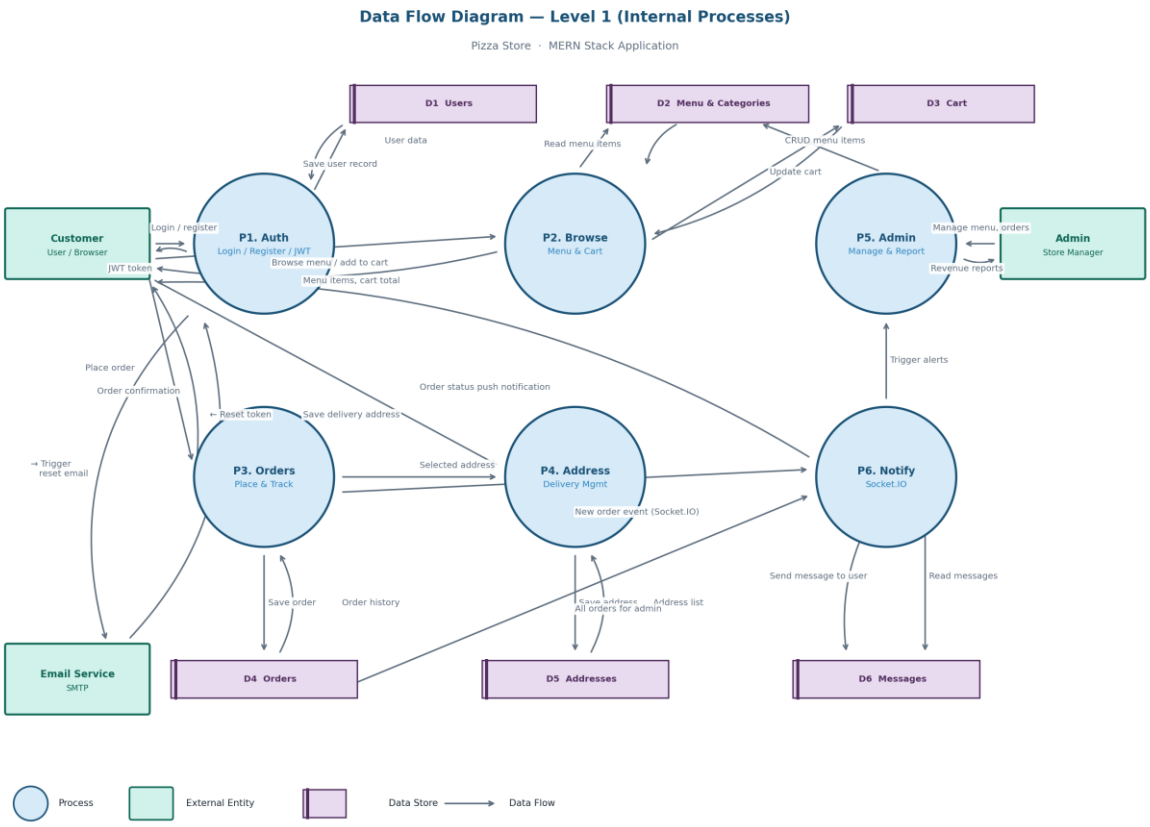


*Figure 3: DFD Level 1 — Internal Processes (Pizza Store)*

## 9. Features Implemented ..... 11

| Feature | Description |
|---|---|
| **Home Page** | Displays pizza products with categories, search bar, and filter tabs. |
| **Authentication System** | Secure registration and login using JWT. Includes forgot/reset password via email. |
| **Product Viewing** | Users can view pizza details: name, description, price, category, and availability. |
| **Cart Functionality** | Real-time cart — add items, update quantity, remove items. Total recalculates instantly. |
| **Order Processing** | Place orders with delivery address, mode (pickup/delivery), and payment choice. |
| **Payment Integration** | Razorpay supports UPI, card, digital wallets. Cash on delivery available. |
| **Coupon System** | Admin creates discount coupons. Customers apply codes at checkout. |
| **Admin Dashboard** | Manage products, categories, users, and orders. Revenue stats displayed. |
| **Real-Time Notifications** | Socket.IO alerts admin instantly on new orders. Status updates pushed live. |
| **Address Management** | Users save multiple delivery addresses and set a default for faster checkout. |
| **Messaging System** | Order-level messages for admin-customer communication. Read/unread tracking. |

## 10. Client-Side Validation ..... 12

Client-side validation verifies user input before sending to the server. On the backend, **express-validator** provides a second layer of defence.

| Form | Validated Fields | Rules Applied |
|---|---|---|
| Registration | Name, Email, Password, Phone | Required, email format, min 6-char password, 10-digit phone |
| Login | Email, Password | Required, valid email format, non-empty password |
| Address Form | House No., Street, City, State, Pin | Required fields, 6-digit pincode validation |
| Order Checkout | Address, Payment Mode | Must select valid address and payment method |
| Password Reset | New Password, Confirm Password | Min 6 chars, both fields must match |

## 11. Routing ..... 12

Frontend uses **React Router v6**. Backend uses **Express.js** to define API route modules.

| Route | Component | Access |
|---|---|---|
| / | HomePage | Public |
| /login | LoginPage | Public |
| /register | RegisterPage | Public |
| /menu | MenuPage | Public |
| /cart | CartPage | Protected |
| /orders | MyOrdersPage | Protected |
| /addresses | AddressPage | Protected |
| /admin | AdminDashboard | Admin Only |
| /admin/orders | AdminOrders | Admin Only |
| /admin/menu | AdminMenuCRUD | Admin Only |

## 12. Frontend–Backend Integration ..... 13

Integration is achieved through **RESTful APIs** using **Axios**. Socket.IO handles real-time order events.

| Step | Action | Detail |
|---|---|---|
| 1 | **User Action** | User clicks Add to Cart or submits a form on the React UI. |
| 2 | **Axios Request** | Redux thunk fires an Axios POST/GET to Express API with JWT in header. |
| 3 | **Auth Middleware** | Express verifies the JWT token. Returns 401 if invalid. |
| 4 | **Controller Logic** | Controller validates input, applies business logic, queries MongoDB. |
| 5 | **Database Query** | Mongoose converts the query to a MongoDB operation and returns data. |
| 6 | **JSON Response** | Backend sends a structured JSON response (success, data, message). |
| 7 | **Redux State Update** | Redux Toolkit updates global store; React re-renders components. |
| 8 | **Socket Event** | For orders, Socket.IO emits an event to notify the admin panel live. |

## 13. API Endpoints Reference ..... 13

### Authentication API — /api/auth

| Method | Endpoint | Description | Access |
|---|---|---|---|
| POST | /register | Register new customer | Public |
| POST | /login | Login + JWT token | Public |
| POST | /logout | Logout user | Public |
| POST | /forgot-password | Send reset email | Public |
| PUT | /reset-password/:token | Reset password | Public |
| GET | /profile | Get user profile | Protected |
| GET | /users | Get all users | Admin |
| PUT | /users/:id/make-admin | Promote to admin | Admin |

### Menu API — /api/menu

| Method | Endpoint | Description | Access |
|---|---|---|---|

| | | | |
|---|---|---|---|
| GET | / | Get all menu items | Public |
| GET | /:id | Get single item | Public |
| POST | / | Create item | Admin |
| PUT | /:id | Update item | Admin |
| DELETE | /:id | Delete item | Admin |

## Orders API — /api/orders

| Method | Endpoint | Description | Access |
|---|---|---|---|
| POST | / | Place new order | Protected |
| GET | /my-orders | User order history | Protected |
| GET | / | All orders | Admin |
| PUT | /:id/status | Update order status | Admin |

## 14. Testing Strategy ..... 15

Testing ensures the application performs correctly and reliably across all layers.

| Testing Type | Tools Used | What Is Tested |
|---|---|---|
| **Unit Testing** | Mocha + Chai | Individual functions, model methods, and utility helpers. |
| **Integration Testing** | Supertest + Mocha | API endpoints — verifies correct responses and error handling. |
| **End-to-End Testing** | Playwright / Cypress | Full user flows: register, login, add to cart, place order. |
| **Manual Testing** | Browser DevTools | UI responsiveness, form validation, and network requests. |
| **Security Testing** | Postman + Manual | JWT expiry, unauthorized access, rate limiting, input sanitization. |

## 15. Application Workflow ..... 15

Complete application flow from user perspective through backend and database:

| Step | Action | Detail |
|---|---|---|
| Step 1 | **Visit Site** | Landing page loads with login form. Unauthenticated users see login/register. |
| Step 2 | **Browse Menu** | After login, user sees the menu with category tabs, search bar, and item cards. |
| Step 3 | **Add to Cart** | Clicking Add triggers an authenticated API call. Cart updates via Redux. |
| Step 4 | **View Cart** | Cart page shows all items, quantities, prices, and total amount. |
| Step 5 | **Select Address** | User selects a saved delivery address or adds a new one. |
| Step 6 | **Payment** | User selects Cash / UPI / Card. Razorpay checkout opens for online payments. |
| Step 7 | **Order Placed** | Order saved in MongoDB. Socket.IO notifies admin dashboard instantly. |
| Step 8 | **Admin Action** | Admin views order and can Accept or Reject it. Status updated accordingly. |
| Step 9 | **User Tracking** | User views order status under My Orders: Pending, Active, or Delivered. |

# 16. Application Screenshots

## 16.1 User Login Page

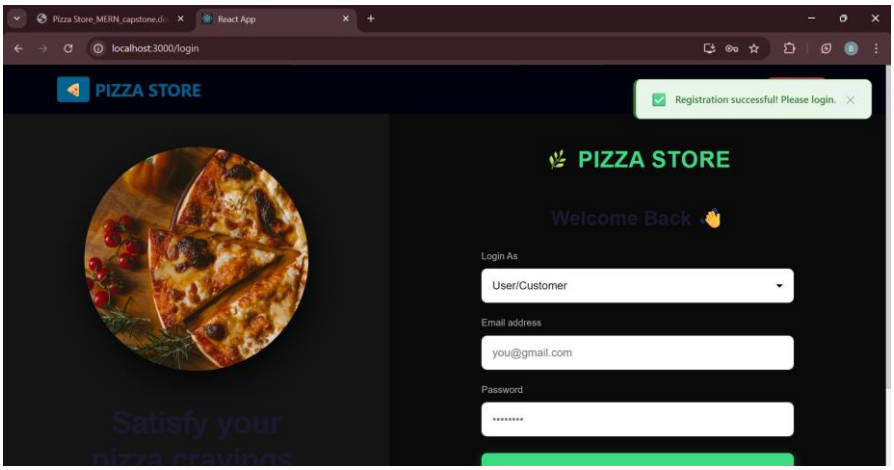


*Figure 14.1 User Login Page*

## 16.2 User Login / Logout

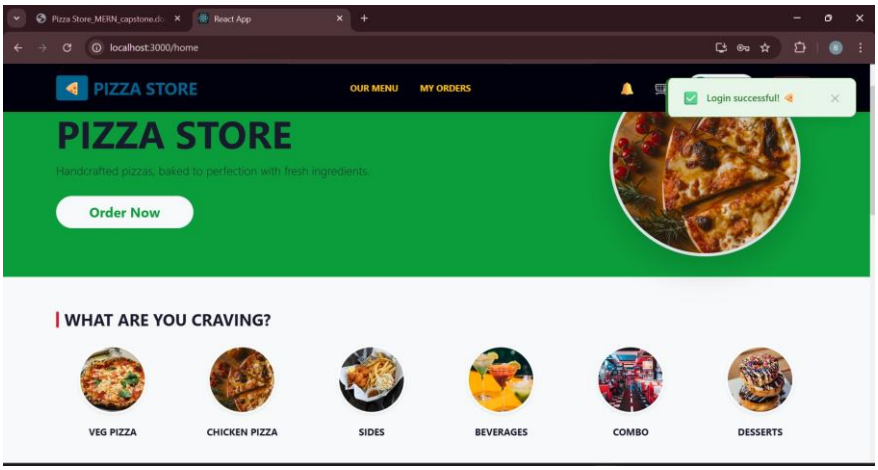


*Figure 14.2 User Login / Logout*

# 16. Application Screenshots

## 16.3 Admin Dashboard

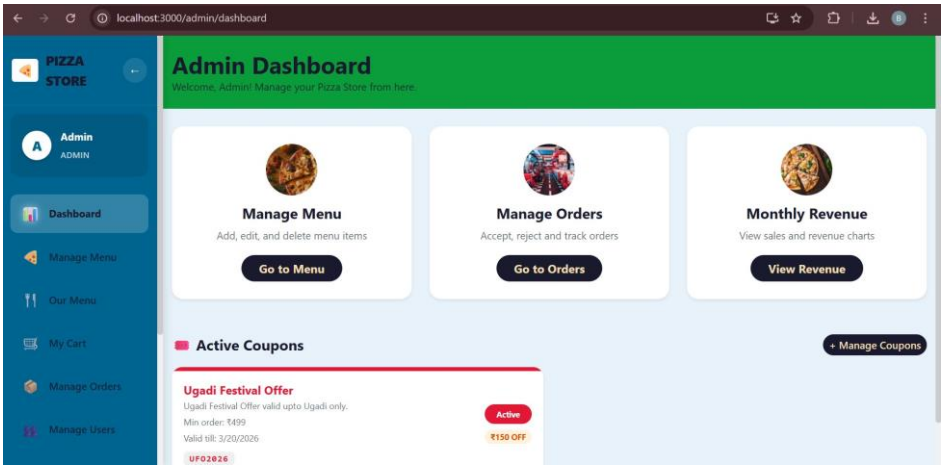


*Figure 14.3 Admin Dashboard*

## 16.4 User Menu — Categories

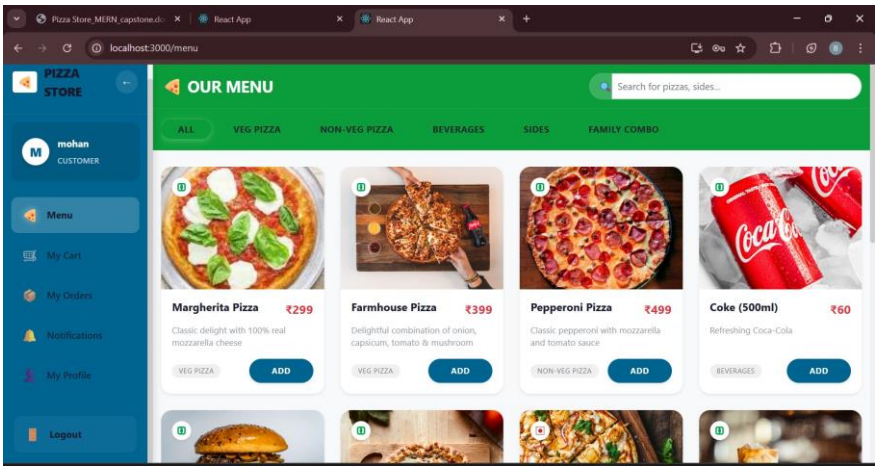


*Figure 14.4 User Menu — Categories*

# 16. Application Screenshots

## 16.5 Responsive / Mobile View

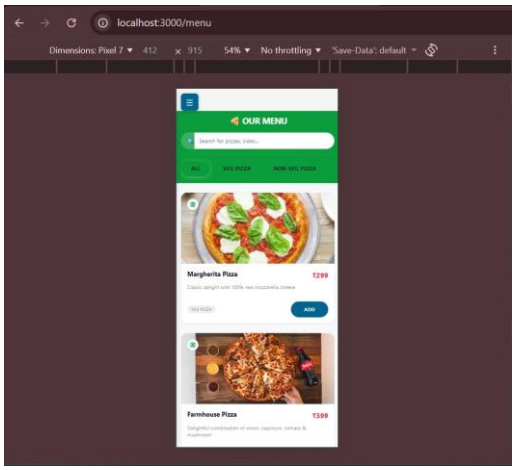


*Figure 14.5 Responsive / Mobile View*

## 16.6 Admin CRUD Operations

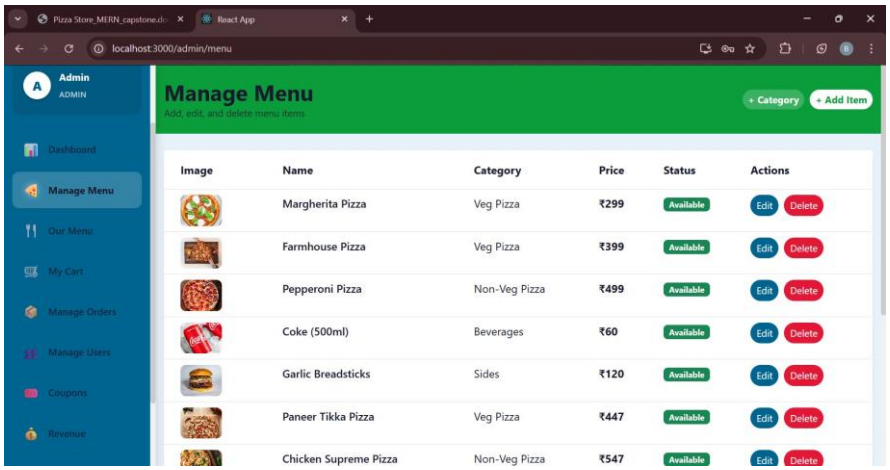


*Figure 14.6 Admin CRUD Operations*

# 16. Application Screenshots

## 16.7 Bill Generation

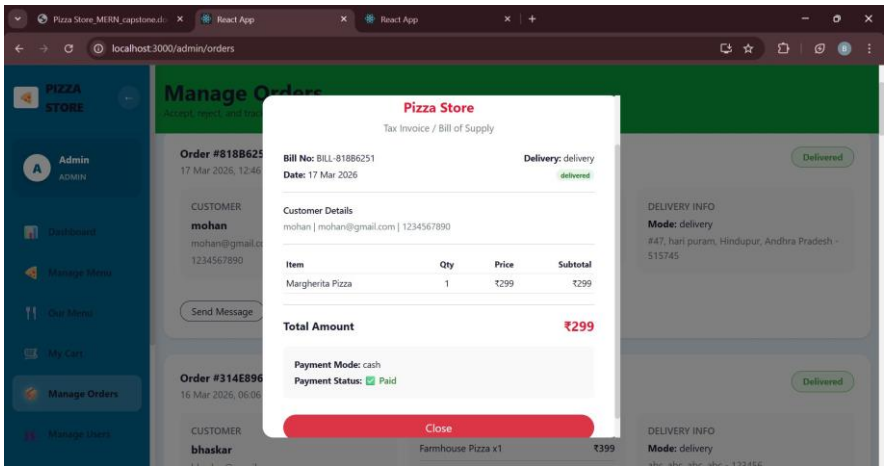


*Figure 14.7 Bill Generation*

## 16.8 Monthly Revenue

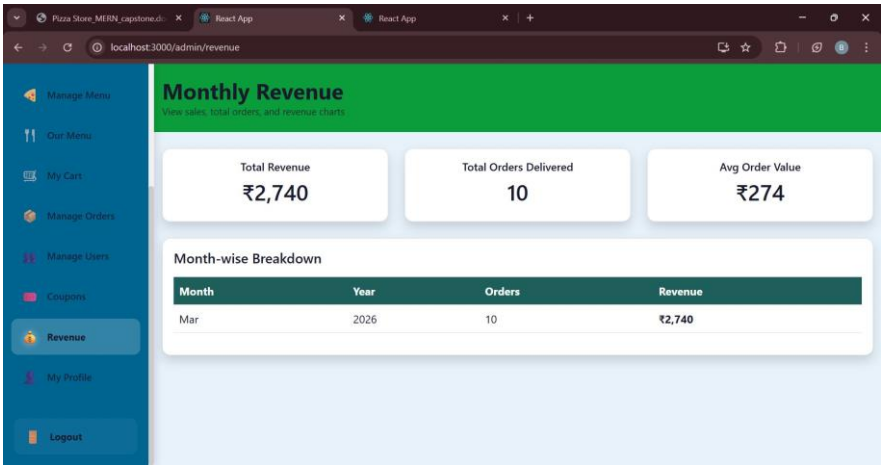


*Figure 14.8 Monthly Revenue*

# 16. Application Screenshots

## 16.9 User Bill / Invoice

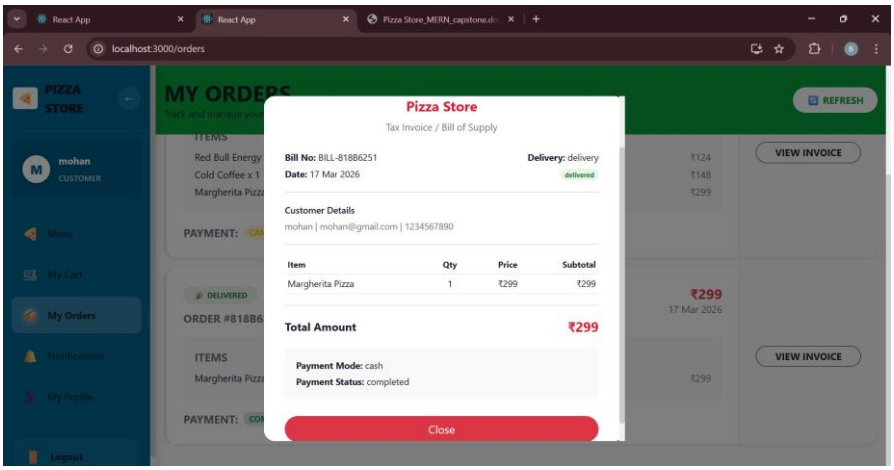


*Figure 14.9 User Bill / Invoice*

## 17. Conclusion ..... 21

The **Pizza Store** application demonstrates the successful development of a modern, full-stack food ordering platform using the **MERN Stack**. Users can easily browse menu items, create accounts, add products to their carts, apply coupons, and place orders through a simple and responsive interface.

The system integrates a **React + Redux** frontend with a **Node.js and Express.js** backend, while **MongoDB** provides efficient and scalable data storage. JWT authentication, role-based access control, input validation, real-time Socket.IO notifications, and Razorpay payment integration contribute to a production-ready user experience.

The **Admin Panel** enables efficient management of products, categories, users, and orders. A structured testing approach using Mocha, Chai, Supertest, and end-to-end testing improves system reliability. This project highlights practical experience in building scalable, secure, and interactive web applications using modern development technologies.

*Submitted by: R Bhaskar Naik | Guided by: Abhishek Sharma | March 2026*